

HermesQA — Báo cáo tiến độ tuần 30/09 → 07/10/2026

Người thực hiện: Phí Thành Dương · Kỳ báo cáo: 30/09/2026 – 07/10/2026 · Mã nguồn: `github.com/zuberg0811/HermesQA` (nhánh `main`, commit `a881536`)

1. Tóm tắt một trang

Tuần này tập trung vào ba việc: (1) **đo lại trung thực** cấu hình kết hợp static + AI vốn trước đây chỉ có số chấm lại offline; (2) **khảo sát Open Code Review của Alibaba** — công cụ cùng bài toán, mở mã nguồn năm 2026 — rút ra 7 đề xuất và **triển khai cả 7**; (3) **thử lần đầu trên pull request thật** (AACR-Bench) thay vì chỉ trên lỗi tự tiêm.

Kết quả chính:

Chỉ số (bộ Python, 35 PR)	Đầu kỳ	Cuối kỳ
F1 cấu hình chỉ AI (<code>11m</code>)	70,1 %	89,5 %
F1 cấu hình kết hợp (<code>both</code> , mặc định sản phẩm)	53,0 % (chấm lại)	87,2 % (chạy thật)
Bảo động giả của <code>11m</code> / <code>both</code>	27 / 61	6 / 8
Nhận xét ghim đúng dòng (<code>11m</code> , độ lệch 0)	33/34	34/34
F1 trên 15 PR thật (AACR-Bench)	chưa đo	1,1 – 2,6 %

Con số cuối cùng là phát hiện quan trọng nhất của tuần: trên code thật, hệ thống gần như im lặng. Điểm 89,5 % phải đọc là "trên bộ đề tự tạo", không phải năng lực thực tế. Báo cáo tiến độ chính (`BAO_CAO_TIEN_DO.md` , 21 trang) đã ghi điều này thành hạn chế rõ ràng.

Khối lượng: 9 commit, 43 file thay đổi (+3 118 / –216 dòng), 35 kiểm thử tự động đều đạt; 1 280 dòng trong `app/` , 615 dòng công cụ đo trong `eval/` , 524 dòng test.

2. Diễn biến theo ngày

05/10 — Đo lại và sửa bước gộp (commit `5b6449a` , `057fead`)

- Chạy lại thật cấu hình `both` trên 35 PR (trước đó chỉ có số chấm lại offline). Phát hiện **bug D3**: bộ lọc nhiễu bandit B101 có test riêng nhưng chưa bao giờ được gọi trong `collect()` — 11 báo động giả lọt qua. Bài học ghi vào báo cáo: *test từng hàm lẻ không chứng minh hàm được gọi; test phải đi qua đúng đường gọi thật*.
- Sửa quy tắc gộp static/AI: static hấp thụ nhận xét AI cùng vị trí nếu AI không chấm nặng hơn; hai vai AI chỉ gộp khi cùng loại lỗi. Hai cách thử trước đều mất 2–3 lỗi thật và bị loại.
- Thêm backend Anthropic (SDK chính thức) sẵn sàng khi có khóa; chưa dùng.
- Số chốt pipeline cũ: `11m` 70,1 % · `both` 66,0 % · đa ngôn ngữ 73,5 / 72,0 %.

05/10 — Khảo sát Open Code Review và triển khai 7 đề xuất (commit `d2c5a0b`)

Chi tiết khảo sát ở mục 3. Bảy đề xuất đã viết thành mã trong cùng ngày:

#	Đề xuất	Hiện thực trong HermesQA
1	Ghim nhận xét bằng đoạn code thay vì số dòng	AI phải trả <code>existing_code</code> ; <code>diff_utils.resolve_anchor</code> tìm lại dòng đúng (khớp nguyên văn → bỏ dấu +/- → dòng dài nhất)
2	Thay self-verify bằng kiểm tra sự thật mặc định giữ	<code>agent.fact_check</code> : AI chỉ được loại nhận xét với căn cứ A (code không tồn tại) hoặc B (code đã xử lý ở chỗ khác); phần mềm kiểm lại bằng chứng; lỗi bảo mật/bug nặng được bảo vệ
3	Danh sách kiểm tra theo loại file, có mục " Không báo "	<code>app/skills/rules/*.md</code> cho Python, JS/TS, Go, Java, Dockerfile, deps, CI, test
4	Chia lô thay vì cắt bớt im lặng	<code>selection.build_batches</code> theo ngân sách ký tự; file bị cắt được liệt kê trong phần "Phạm vi review"
5	Loại trừ mặc định + chặn file bí mật	<code>.env</code> , khóa, lockfile, file sinh tự động không bao giờ gửi cho AI; dòng gitleaks đánh dấu được che
6	Dấu vân tay nhận xét giữa các lần chạy	<code>postprocess.fingerprint</code> (file + loại + nội dung code); bảng <code>posted_comments</code> ; không đăng lại nhận xét cũ
7	Nâng cấp đánh giá	<code>score.py --tolerance 0</code> , <code>judge_fp.py</code> (giám khảo AI phân loại báo động giả), <code>aacr_eval.py</code> (pilot PR thật)

Kèm theo: `pipeline.py` dùng chung cho CLI và worker (trước đây hai đường chạy khác nhau), 21 test mới.

06/10 — Đo pipeline mới, sửa theo số đo (commit `2133adf` , `87c12e2`)

- Hạn mức miễn phí của Gemini (500 lượt/ngày/model) cạn giữa chừng nên bộ Python phải chia sang ngày 07/10. Pilot PR thật chạy bằng model cùng họ (`gemini-3.5-flash-lite`).
- Đọc dữ liệu trung gian (`_meta`) phát hiện **fact-check lý luận vòng**: AI loại 4 nhận xét *đúng* với "bằng chứng" chính là dòng bị báo lỗi. Thêm ràng buộc cơ học: bằng chứng căn cứ B phải là dòng khác dòng bị báo.
- Pilot PR thật lần đầu chạy **sai dữ liệu**: hiểu ngược hai trường commit của AACR-Bench (`pr_source_commit` thực ra là nhánh đích) nên review diff đảo chiều hoặc kèm hàng nghìn dòng không thuộc PR. Phát hiện nhờ đối chiếu số dòng với dataset và GitHub API; bỏ toàn bộ lần chạy đó, sửa lấy diff `base...head` .
- Phát hiện **240/275 cảnh báo static trên PR thật nằm ngoài phần sửa** (GitHub cũng không cho đăng comment ở đó) → thêm bộ lọc "chỉ trong phạm vi thay đổi" vào cả CLI lẫn worker.
- Bền vững hơn với API: chấp nhận JSON có xuống dòng thật, gọi lại khi JSON hỏng, tự đợi khi gặp hạn mức theo phút.

07/10 — Chạy lại đủ, chốt số, cập nhật tài liệu (commit `8e87c2e` , `869687c` , `a881536`)

- Chạy lại toàn bộ 4 cấu hình bộ Python và 2 cấu hình đa ngôn ngữ với cùng model cũ (`gemini-3.1-flash-lite`) để so sánh công bằng; ~420 lượt gọi.
- Đọc 15 báo động giả của `both` : 7 cái là **hai công cụ static báo cùng một lỗi ở cùng dòng** với hai nhãn khác nhau → gộp mọi cảnh báo static cùng dòng thành một comment. F1 `both` 81,4 → 87,2 % , đổi lại mất 1 lỗi thật vì nhãn của cụm lấy theo cảnh báo chính (đã ghi là hạn chế).
- Giám khảo AI phân loại báo động giả (model khác model sinh, 3 phiếu): Python 20/20 là nhận xét đúng ngoài đáp án; đa ngôn ngữ 5/6 đúng, **1 bìa** (4 % tổng nhận xét).
- Cập nhật `BAO CAO TIEN DO.md` (nhóm E, bảng 7.6), tài liệu đề xuất (mục 9 kết quả), trang tổng quan; thêm `tools/md2pdf.py` để mọi PDF cùng định dạng.

3. Nghiên cứu giải pháp của Alibaba: Open Code Review

Là gì: công cụ review pull request bằng AI do Alibaba dùng nội bộ hai năm, mở mã nguồn năm 2026 (Go, Apache-2.0, hơn 43 000 sao GitHub). Đi kèm bài báo và bộ dữ liệu **AACR-Bench**: 200 PR thật, 10 ngôn ngữ, 1 505 lỗi do 80 kỹ sư xác nhận. Đã đọc toàn bộ phần chọn file, định vị nhận xét, lời nhắc, bộ lọc, luật theo ngôn ngữ, và bài báo.

Điểm mạnh (và HermesQA đã học gì)

Điểm mạnh của OCR	Vì sao quan trọng	HermesQA
Định vị nhận xét bằng đoạn code (<code>existing_code</code>) rồi dò lại vị trí bằng cửa sổ trượt	Số dòng do AI đọc rất hay sai; đoạn code thì không	Đã làm (đề xuất 1): 100 % nhận xét ghim được, đúng dòng
Bộ lọc "fact-checker" mặc định <i>giữ</i> , chỉ loại khi có căn cứ cụ thể, bảo vệ nhóm lỗi nặng	Self-verify kiểu "viết lại cho đúng" làm mất lỗi thật — chúng tôi đo được điều này	Đã làm (đề xuất 2), có thêm kiểm chứng cơ học mà OCR không có
Luật theo ngôn ngữ kèm "Do not report"	Giảm nhận xét đúng-nhưng-vô-ích — nguồn FP lớn nhất	Đã làm (đề xuất 3): FP giảm 4–5 lần
Chọn file tất định (6 cổng, loại trừ mặc định, đường dẫn bí mật), gom file có liên quan vào cùng một lượt	Không gửi bí mật cho AI; AI thấy đủ ngữ cảnh	Đã làm (đề xuất 4, 5)
Đánh giá trên PR thật với giám khảo AI bỏ phiếu	Lỗi tự tìm quá dễ; số liệu thật mới có ý nghĩa	Đã làm pilot 15 PR (đề xuất 7) — và nó cho thấy khoảng cách lớn
Vận hành chín muồi: GitHub Action một dòng, nén bộ nhớ vòng lặp AI, nhiều nhà cung cấp model	Đã dùng thật ở quy mô lớn	Chưa cần ở giai đoạn đồ án

Điểm yếu / khác biệt (và HermesQA giữ gì của riêng mình)

Điểm yếu của OCR	Nhận định	HermesQA
Không có công cụ static — hoàn toàn dựa vào AI	Lỗi bảo mật kinh điển (SQL injection, secret, shell=True) có thể bắt bằng luật tất định, rẻ và không bịa	Giữ 5 công cụ static trong sandbox cách ly; static ngang AI ở nhóm bảo mật (12/13)
Recall thấp ngay cả với model mạnh: Claude 4.5 Sonnet agent chỉ P 39,9 / R 10,1 / F1 16,1 trên AACR-Bench	Bài toán review PR thật khó hơn nhiều so với benchmark lỗi tiềm; không công cụ nào "giải xong"	Số pilot của chúng tôi (R 1,5–3 %) càng thấp hơn vì model nhỏ; cần model mạnh hơn mới so được
Phụ thuộc hoàn toàn vào chất lượng model và lời nhắc dài; không có lớp kiểm chứng cơ học	Fact-checker của OCR cũng là AI, tự kiểm AI	HermesQA kiểm lại bằng chứng bằng phần mềm (bằng chứng phải tồn tại trong diff, không được là chính dòng bị báo) — đo được nó chặn 4 lần loại sai
Không có sandbox; agent có công cụ đọc file tự do	Phù hợp nội bộ Alibaba, rủi ro khi chạy mã lạ	Giữ sandbox Docker không mạng, chỉ đọc
Một vai duy nhất	Đơn giản, nhưng bỏ sót góc nhìn test/DevOps	Giữ ba vai SE / QA / DevOps theo loại file
Benchmark của họ là tự xây, tự chấm bằng AI	Có thiên vị "đồng ý với chính mình" (chúng tôi gặp đúng điều này với giám khảo nhỏ)	Ghi rõ trong hạn chế; nên dùng model khác họ làm giám khảo

Kết luận của khảo sát: OCR giải tốt phần *định vị, lọc và vận hành*; HermesQA học phần đó và giữ phần *lai static + sandbox + nhiều vai* làm điểm khác biệt. Điều cả hai bên cùng chứng minh: trên PR thật, AI review còn xa mức "thay người".

4. Số liệu chi tiết cuối kỳ

Bộ Python, 35 PR, 36 lỗi; model `gemini-3.1-flash-lite`. "Đúng dòng" = chấm ở độ lệch 0 dòng.

Cấu hình	TP	FP	FN	Precision	Recall	F1	F1 đúng dòng	Giây / token mỗi PR
<code>static</code> (chấm lại lần chạy gốc)	21	9	15	70,0 %	58,3 %	63,6 %	54,5 %	101 s · 0
<code>llm</code> — kiểm tra sự thật	34	6	2	85,0 %	94,4 %	89,5 %	89,5 %	18,5 s · 6 355
<code>llm_noverify</code>	34	6	2	85,0 %	94,4 %	89,5 %	89,5 %	12,2 s · 4 532
<code>llm_legacyverify</code> (self-verify cũ)	33	11	3	75,0 %	91,7 %	82,5 %	82,5 %	18,6 s · 8 656
<code>both</code> — mặc định sản phẩm	34	8	2	81,0 %	94,4 %	87,2 %	76,9 %	118 s · 6 551

Bộ đa ngôn ngữ (18 PR): `llm` 85,7 % (trước 73,5), `both` 81,8 % (trước 72,0), recall 100 %.

PR thật (AACR-Bench, 15 PR Python ≤ 210 dòng, 68 vấn đề; model `gemini-3.5-flash-lite`):

Cấu hình	Nhận xét	Trúng	Precision	Recall	F1
<code>llm</code>	8	1	12,5 %	1,5 %	2,6 %
<code>both</code> như cũ	286	2	0,7 %	2,9 %	1,1 %
<code>both</code> , chỉ giữ cảnh báo trong phạm vi thay đổi	43	1	2,3 %	1,5 %	1,8 %

Ba kết luận: (1) báo động giả giảm 4–5 lần chủ yếu nhờ danh sách "Không báo", nhưng danh sách được viết sau khi nhìn thấy bộ lỗi tiềm nên có nguy cơ "học thuộc đề"; (2) self-verify kiểu cũ làm xấu kết quả, fact-check mới an toàn nhưng chưa chứng minh được lợi ích với model nhỏ; (3) trên PR thật hệ thống gần như im lặng — cần thử model mạnh hơn.

5. Lỗi tìm được nhờ đo đạc (ghi lại để không lặp)

Lỗi	Cách phát hiện	Bài học
D3: bộ lọc nhiễu có test nhưng chưa được gọi	Chạy thật <code>both</code> , thấy 11 B101 lọt	Test phải đi qua đường gọi thật
Fact-check lý luận vòng loại 4 nhận xét đúng	Đọc file <code>_meta</code> ghi lý do loại	Mọi quyết định của AI cần dữ liệu trung gian để truy vết
Diff PR thật ngược chiều / thừa hàng nghìn dòng	Số số dòng diff với dataset và GitHub API	Kiểm tra kích thước đầu vào trước khi tin kết quả
240/275 cảnh báo static ngoài phần sửa	Phân loại vị trí finding theo hunk	Benchmark nhỏ che giấu vấn đề chỉ lộ trên repo thật
Hai công cụ static báo cùng lỗi thành 2 nhận xét	Rà tay 15 báo động giả	Đọc từng FP, không chỉ nhìn tổng

6. Kế hoạch tuần tới

1. **Website demo** sau khi hoàn thiện: trang giới thiệu, đăng nhập GitHub, xem kết quả review của các PR đã chạy, nút "review thử" trên repo mẫu. Phần chạy review (worker + sandbox Docker) vẫn phải ở máy chủ riêng (VPS hoặc Cloud Run); Firebase chỉ phù hợp cho phần hiển thị (Hosting, Auth, Firestore cho trạng thái thời gian thực) — xem nhận định riêng gửi kèm.
2. Thử model mạnh hơn (Claude hoặc Gemini Pro) trên cả ba bộ, đặc biệt 15 PR thật, để tách "im lặng do model" khỏi "im lặng do lỗi nhắc".
3. Dùng model khác họ làm giám khảo phân loại báo động giả, hoặc rà tay một mẫu.
4. Test luồng GitHub App thật trên một repo demo (cần credential).

7. Phụ lục: danh sách commit trong kỳ

Ngày	Commit	Nội dung
05/10	5b6449a	Sửa bước gộp static/LLM theo kết quả eval, lọc nhiễu bandit B101, cập nhật báo cáo
05/10	057fead	Chạy lại both thật, nối bộ lọc B101 vào collect() (bug D3), backend Anthropic, đề xuất từ Open Code Review
05/10	d2c5a0b	Triển khai 7 đề xuất từ khảo sát Open Code Review
06/10	2133adf	Chặn fact-check lý luận vòng, lọc static theo hunk, diff AACR đúng chiều
06/10	87c12e2	Đề xuất OCR: mục 9 kết quả sau triển khai, PDF
07/10	8e87c2e	Gộp mọi finding static cùng dòng; gọi lại JSON hỏng kèm thông báo lỗi; README eval
07/10	869687c	Đề xuất OCR mục 9: số đo cuối
07/10	ac611ea	gitignore thư mục rescore
07/10	a881536	Báo cáo tiến độ nhóm E, bảng 7.6, tools/md2pdf.py, PDF

Tài liệu đi kèm trong repo: BAO_CAO_TIEN_DO.md (+PDF, 21 trang), docs/DE_XUAT_CAI_TIEN_TU_OPEN_CODE_REVIEW.md (+PDF, 9 trang), eval/README.md.